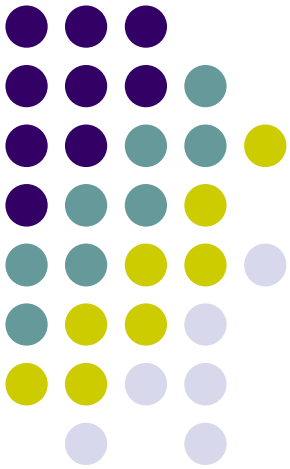
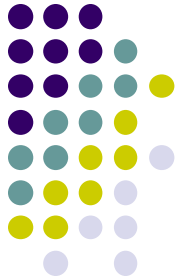
# Chapter 2
# Processes and Threads



2.1 Processes
2.2 Threads
2.3 Interprocess communication
2.4 Classical IPC problems
2.5 Scheduling

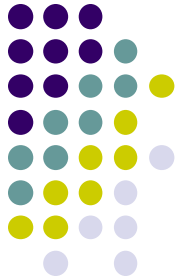
# 2.1 Process



1. Sequential Execution

- Sequential顺序性
  - the order of program structure (maybe switch or loop)
- Close封闭性
  - Occupy all resource alone
- Reappear可再现性
  - if the initial condition be same, the result be same

2. Concurrency Execution: Lose all the characters



- Concurrency → Process
  - Multiprogramming
  - Problem:

    compete the resource;

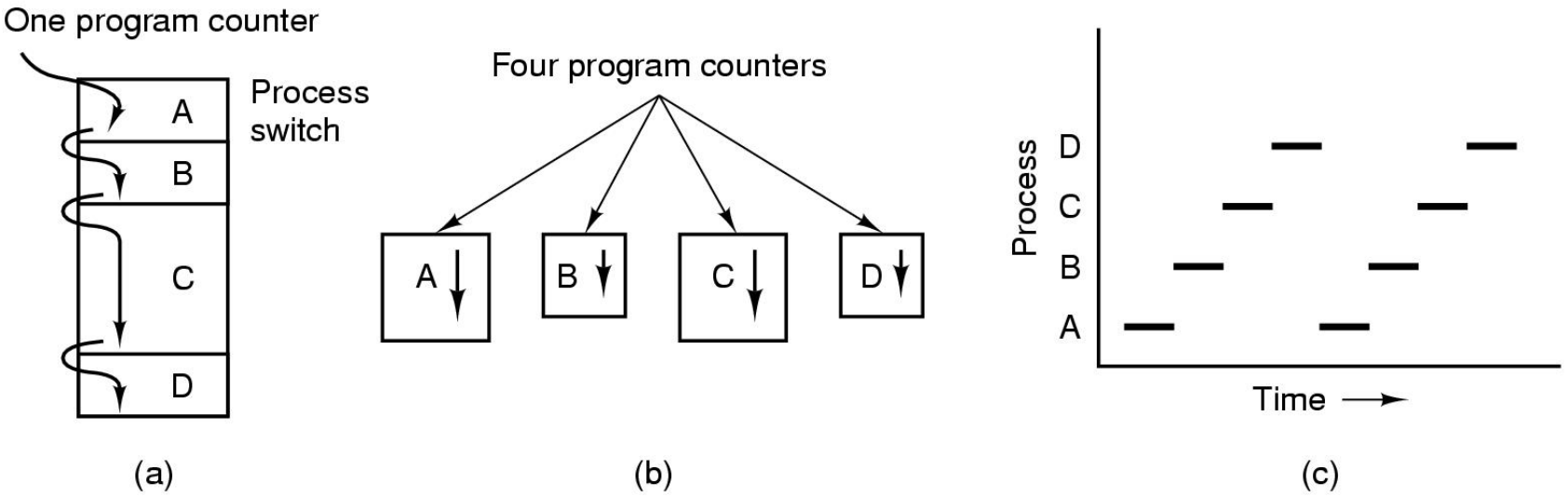
    communication;

    collaboration …

  - Solution: process concept

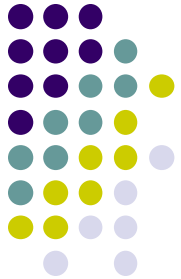
# The Process Model

- In the process model, all runnable software is organized as a collection of processes.




a) Multiprogramming of four programs
b) Conceptual model of 4 independent, sequential processes
c) Only one program active at any instant

# Process Concept



- from MULTICS and CTSS/360 system (task)
- An abstraction of a running program
- a program in execution

- 进程，即是一个具有一定独立功能的程序关于某个数据集合的一次活动，是系统进行资源分配和调度的独立单位，在Linux内又称任务（Task）

- Analogy:

  birthday cake recipe Program

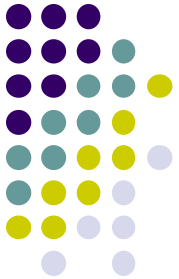
  activity (reading, fetching, baking) Process

# Differences between process and program



1. Program: the collection of instruction, static concept; Process: describe concurrency, dynamic concept
2. Process include program, data, and PCB
3. Process: temporary; Program: permanent
4. A program can be the execution program of multiple process;  also, a process can call multiple programs.
5. Process can create other process in it.

# Process Creation (when?)

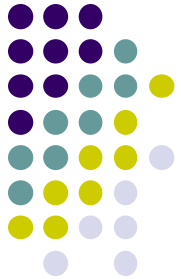


- Principal events that cause process creation
  1. System initialization (reboot)
  2. Execution of a process creation system call (fork())
  3. User request to create a new process
     Command line or click an icon
  4. Initiation of a batch job

- Foreground processes are those that interact with users and perform work for them

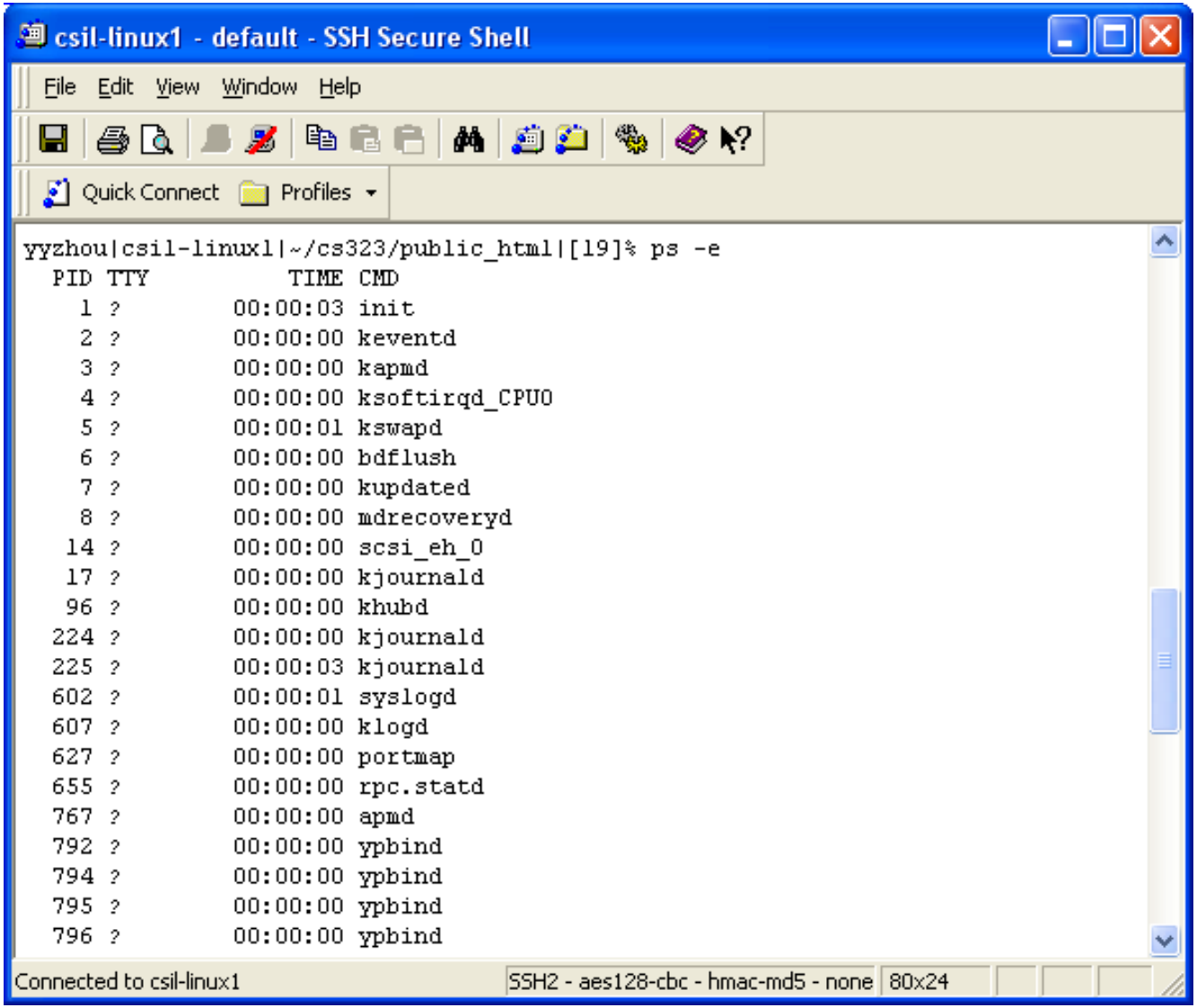
- Background processes that handle some incoming request are called **daemons**

# Process Creation

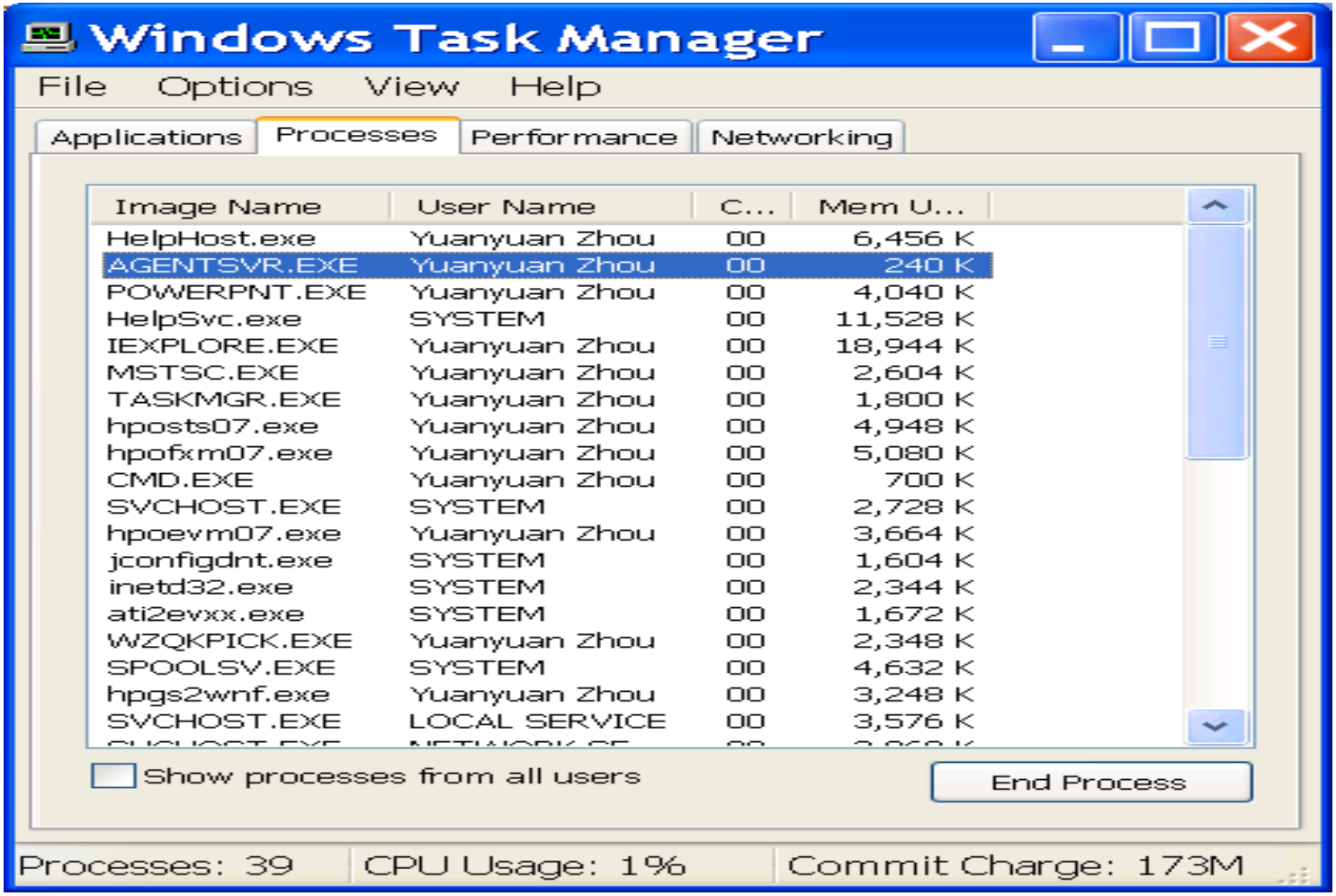


- In UNIX, a fork() system call is used to create a new process.
  - Initially, the parent and the child have the same memory image, the same environment strings, and the same open files.
  - The execve() system call can be used to load a new program.
  - But the parent and child have their own distinct address space.
- In Windows, CreateProcess handles both creation and loading the correct program into the new process.
- How to list the running processes?
  - In UNIX, use the ps command
  - In Windows, use the task manager

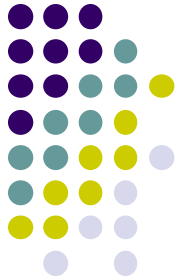
# Unix Example: ps



# Windows Task Manager

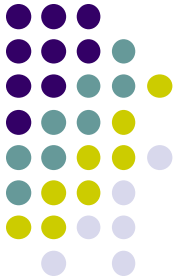


# Process Termination (when?)



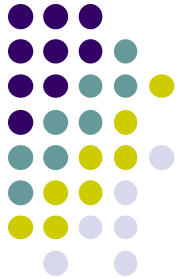
- Conditions which terminate processes
  1. Normal exit (voluntary)
     - Exit in UNIX and ExitProcess in Windows.
  2. Error exit (voluntary)
     - Example: Compiling errors.
  3. Fatal error (involuntary)
     - Example: Core dump
  4. Killed by another process (involuntary)
     - Kill in UNIX and TerminateProcess in Windows.

# Process Hierarchies



- Parent creates a child process, child processes can create its own processes
- Forms a hierarchy (init)
  - UNIX calls this a “process group”
- Windows has no concept of process hierarchy
  - all processes are created equal

# Process States (1)

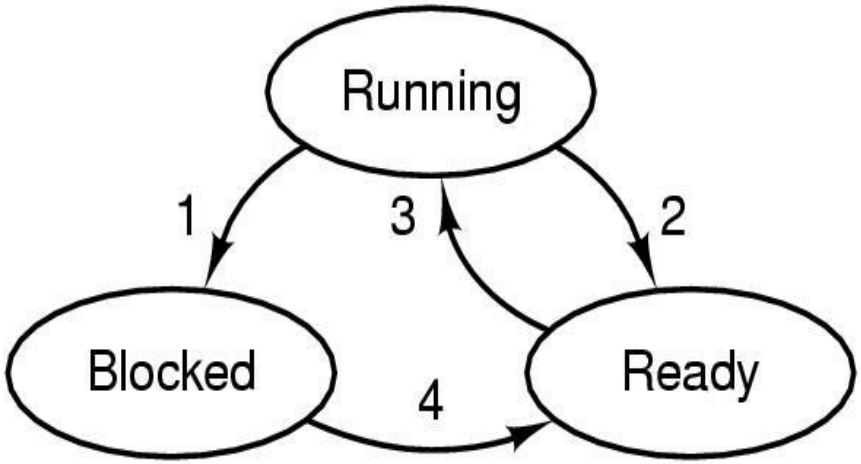


- basic process states
  - Running - using the CPU
  - Ready - runnable (in the ready queue)
  - Blocked - unable to run until an external event occurs
    e.g., waiting for a key to be pressed

不同系统设置的进程状态数目不同

# Process States (2)

Transitions between process states:

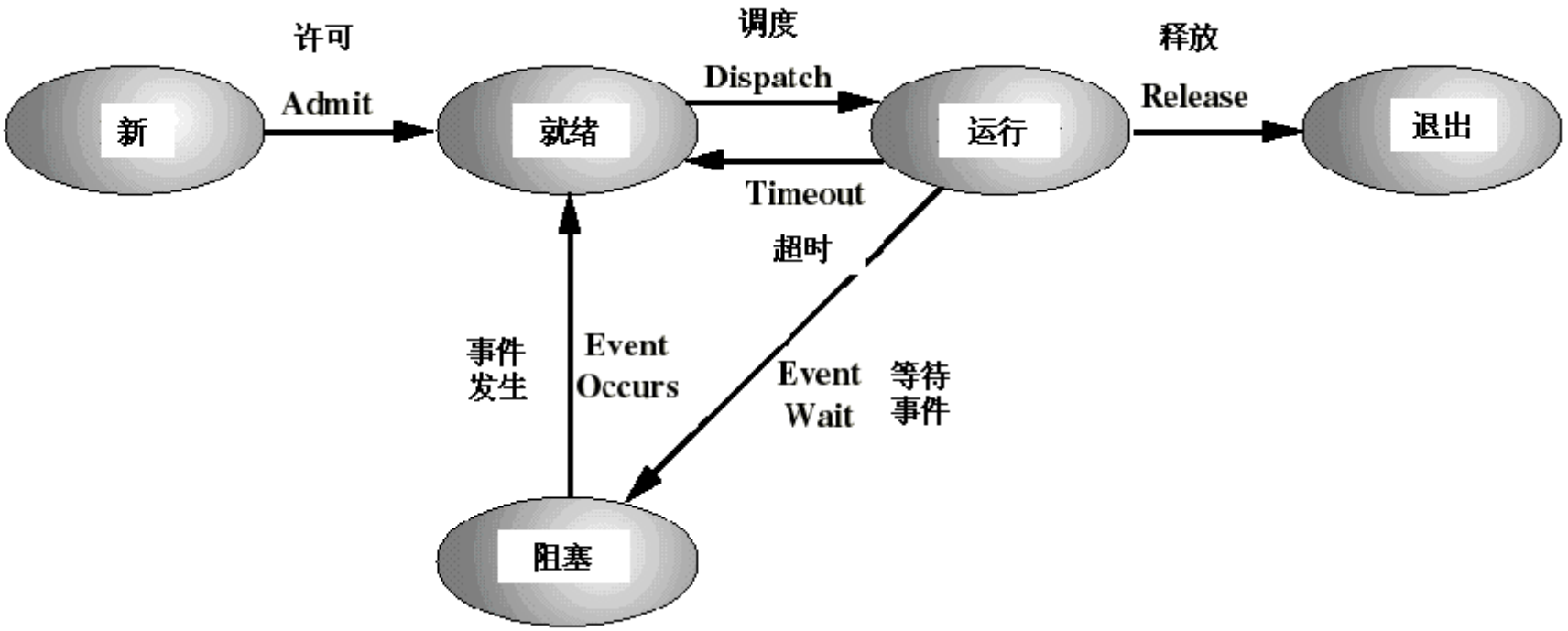



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

# Process States (3)

- Other process states

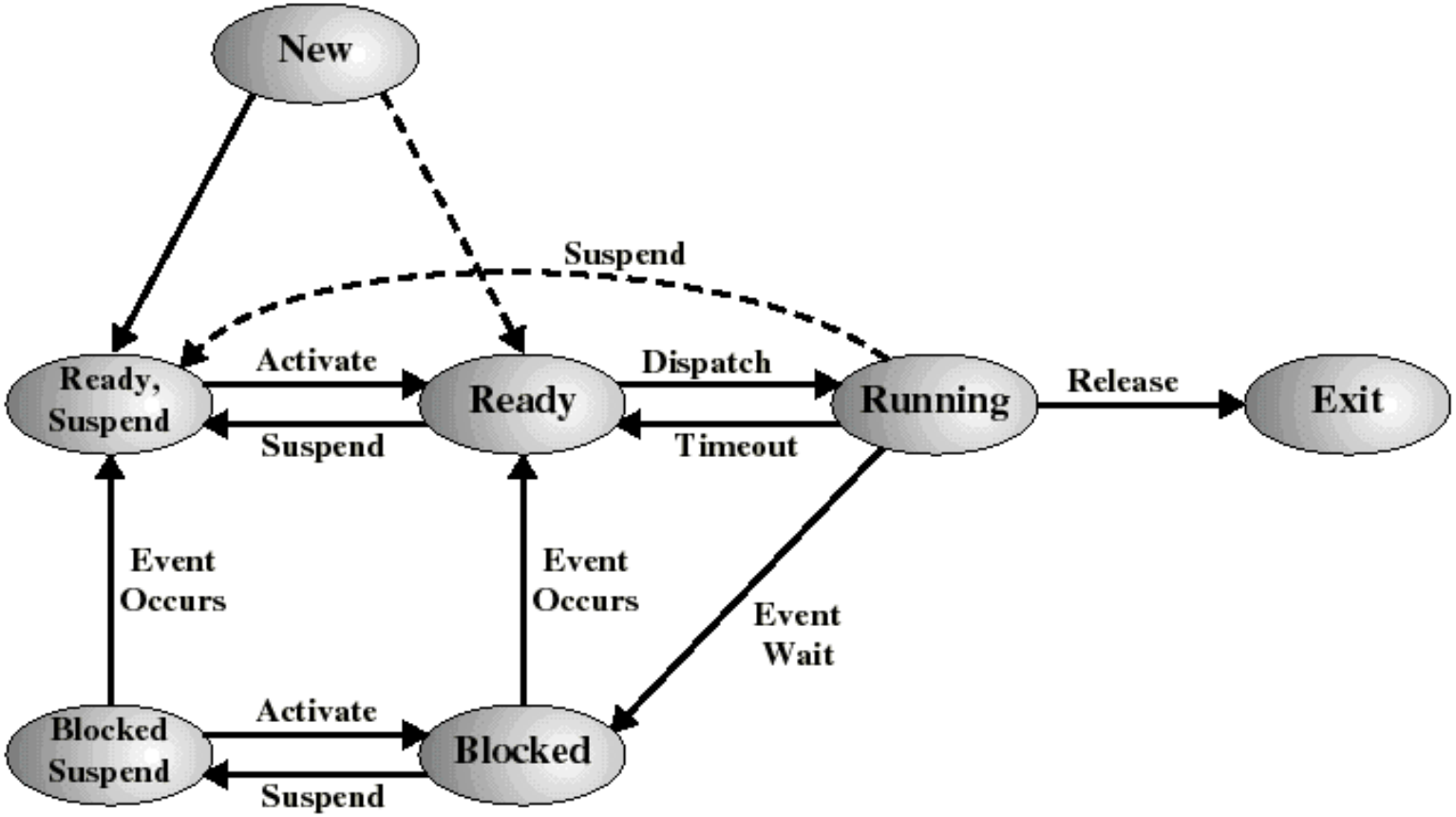
  New state, Exit state:



# Process States (4)

Suspend :

- Blocked, suspend state
- Ready, suspend



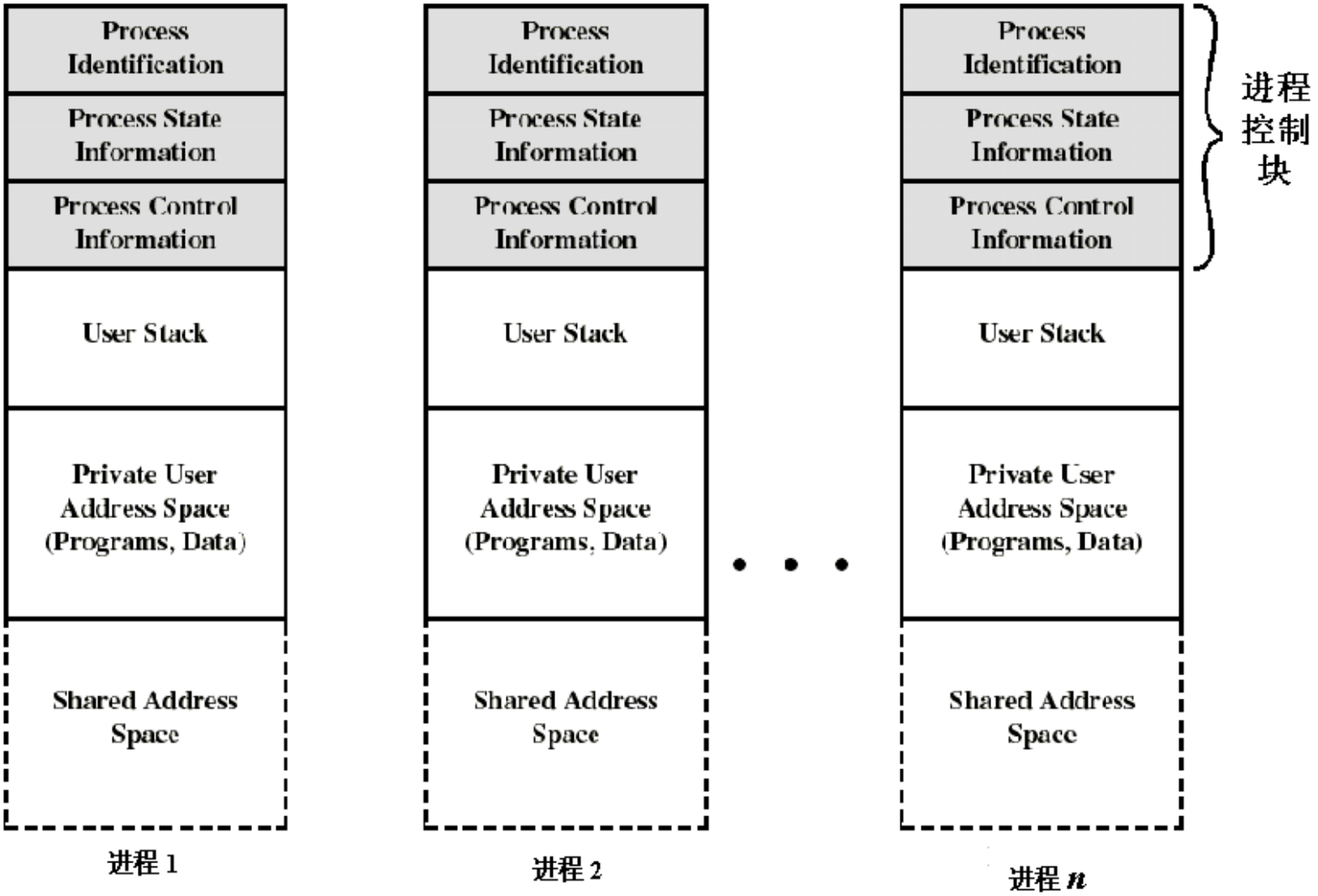
# Implementation of Processes(1)



- What is in a process? [Process image]
  - User program
  - User data
  - Stack (procedure call and parameter passing)
  - PCB [process control block]

    (process attribute)
    - In kernel
    - 用户进程不能直接访问、修改自己的PCB

# 在虚存中的进程映象


Process Identification
Process State Information
Process Control Information
User Stack
Private User Address Space (Programs, Data)
Shared Address Space
进程控制块
进程 1
进程 2
进程 n

# Implementation of Processes(2)



| Process management | Memory management | File management |
|---|---|---|
| Registers | Pointer to text segment | Root directory |
| Program counter | Pointer to data segment | Working directory |
| Program status word | Pointer to stack segment | File descriptors |
| Stack pointer | | User ID |
| Process state | | Group ID |
| Priority | | |
| Scheduling parameters | | |
| Process ID | | |
| Parent process | | |
| Process group | | |
| Signals | | |
| Time when process started | | |
| CPU time used | | |
| Children's CPU time | | |
| Time of next alarm | | |

Some of the fields of a typical process table entry

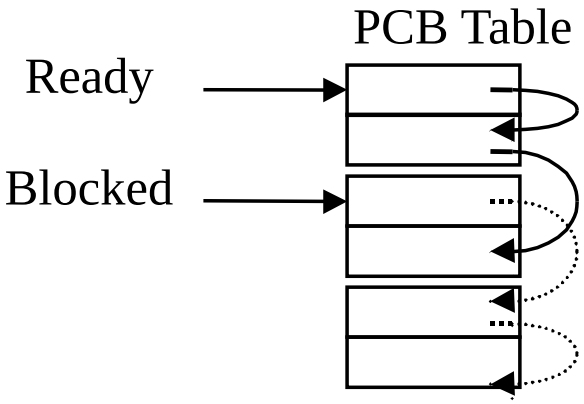
# Implementation of Processes(3)

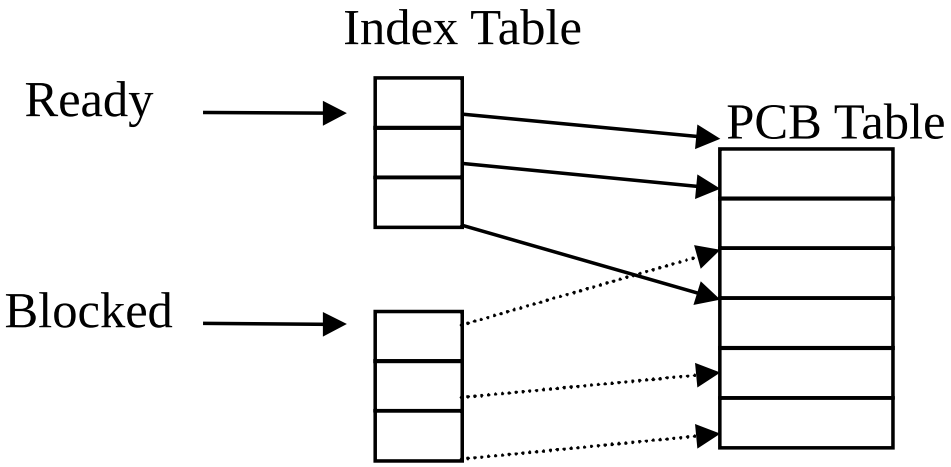


- **Process Context**
  - The static description of the whole execution process
  - User Context, Register Context, System Context
- **Context Switch**
  - Switch CPU from one process to another
  - Performed by scheduler (chapter 2.5)
  - It includes:
    - save PCB state of the old process;
    - load PCB state of the new process;
    - Flush memory cache;
    - Change memory mapping;
  - Context switch is expensive(1-1000 microseconds)
    - No useful work is done (pure overhead)
    - Can become a bottleneck

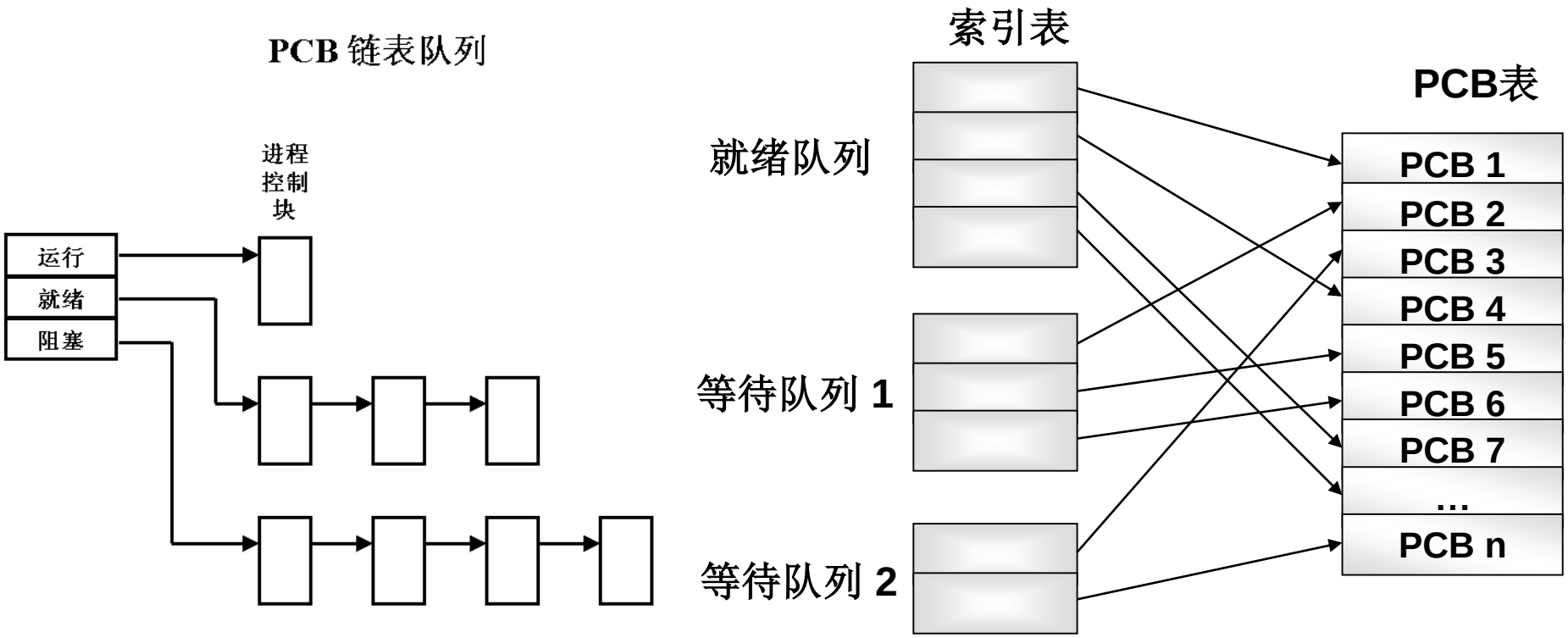
# Implementation of Processes(4)

- Process table or PCB table
  - OS maintains a process table, each entry is PCB
  - The size of PCB table determine the concurrency degree of system
- Two organization form:
  - Link
  - Index

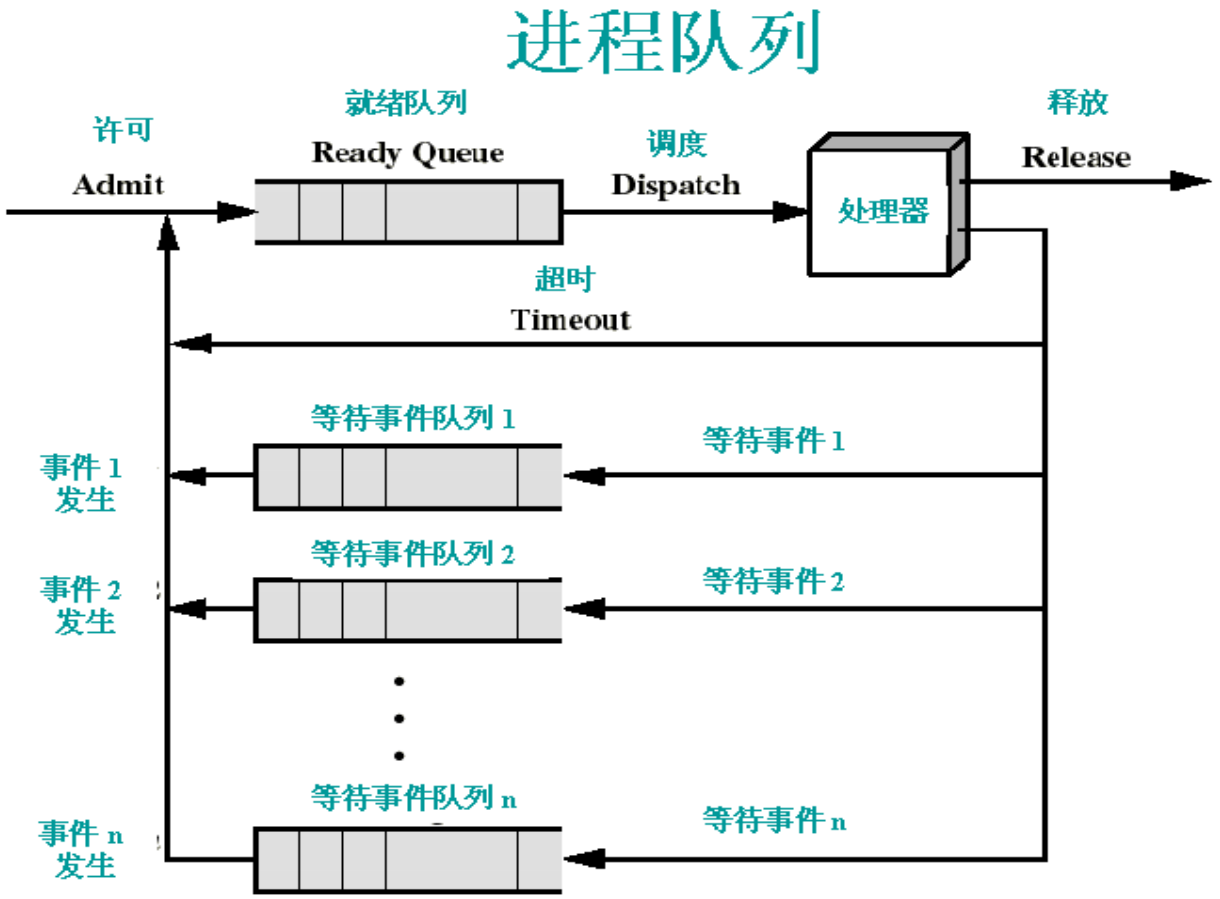


# Implementation of Processes(5)

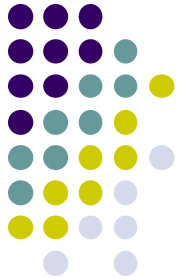


# Implementation of Processes(6)



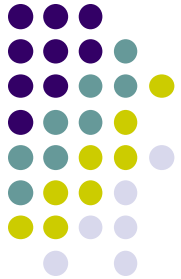

- 就绪队列无优先级（例：FIFO）
- 当事件n发生，对应队列移进就绪队列

# Summary(1)



- What is process
- Differences between process and program
- Process states and model
- What information in a process
- Process Context switch

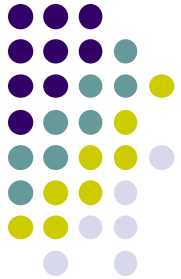
# 2. Thread



- Basic attribute of **Process**:
  - 1. Resource allocate unit
  - 2. Scheduler unit
- OS must do:
  - Create process
  - Terminate process
  - Switch Process

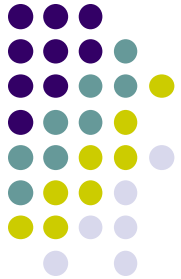
  Spend much time and space, restrict the concurrency.

# Thread concept



- 如何能使多个程序更好地并发执行，同时又尽量减少系统的开销，成为设计操作系统的重要目标。
- 将进程的两个基本属性分开，由操作系统分开处理：
  - 对于作为调度和分派的基本单位，不同时作为拥有资源的单位，以做到“轻装上阵”
  - 对于拥有资源的基本单位，又不对之进行频繁的切换
- A thread – **lightweight process (LWP)**
  - Thread: a basic unit of CPU utilization.
  - Process: a basic unit of resource allocation.

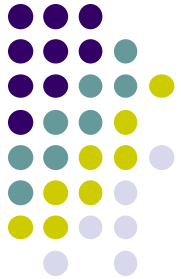
# Thread concept



- 原进程PCB的内容分成两部分：
  - 1. 描述进程资源和空间的部分；
  - 2. 描述执行现场、状态及调度的部分。

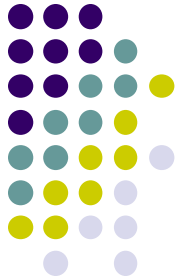
  将第2部分内容作为线程控制块TCB的内容，且一个进程内允许多个线程存在。
- 新进程描述为：
  - 一个独立的进程空间，可装入进程映像；
  - 一个独立的进程相关联的执行文件；
  - 进程所用的系统资源；
  - 一个或多个线程。（进程在创建时一般同时创建好第一个线程，其他线程按需要由用户程序请求创建）

# Thread concept



- 线程不拥有系统资源，只有其运行所必需的一些数据结构：TCB, a program counter, a register set, and a stack
  它与该进程内其它线程共享该进程所拥有的全部资源
  - 堆栈和寄存器用来存储线程内的局部变量；
  - 线程控制块TCB用来说明线程存在的标识、记录线程属性和调度信息
- A traditional (**heavyweight**) process has a single thread of control
- If the process has multiple threads of control, it can do more than one task at a time. This situation is called **multithreading**

# Thread vs Process

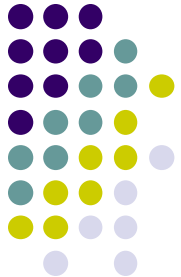


① **进程是资源分配的基本单位**，所有与该进程有关的资源分配情况，如打印机、I/O缓冲队列等，均记录在进程控制块PCB中，进程也是分配主存的基本单位，它拥有一个完整的虚拟地址空间。而线程与资源分配无关，它属于某一个进程，并与该进程内的其它线程一起共享进程的资源

② 不同的进程拥有不同的虚拟地址空间，而同一进程中的多个线程共享同一地址空间

③ 进程调度的切换将涉及到有关资源指针的保存及进程地址空间的转换等问题。而线程的切换将不涉及资源指针的保存和地址空间的变化。所以，线程切换的开销要比进程切换的开销小得多

# Thread vs Process

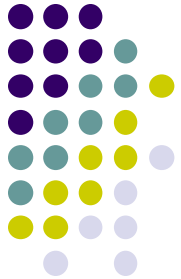


④ 进程的调度与切换都是由操作系统内核完成，而线程则既可由操作系统内核完成，也可由用户程序进行

⑤ 进程可以动态创建进程。被进程创建的线程也可以创建其它线程

⑥ 进程有创建、执行、消亡的生命周期。线程也有类似的生命周期

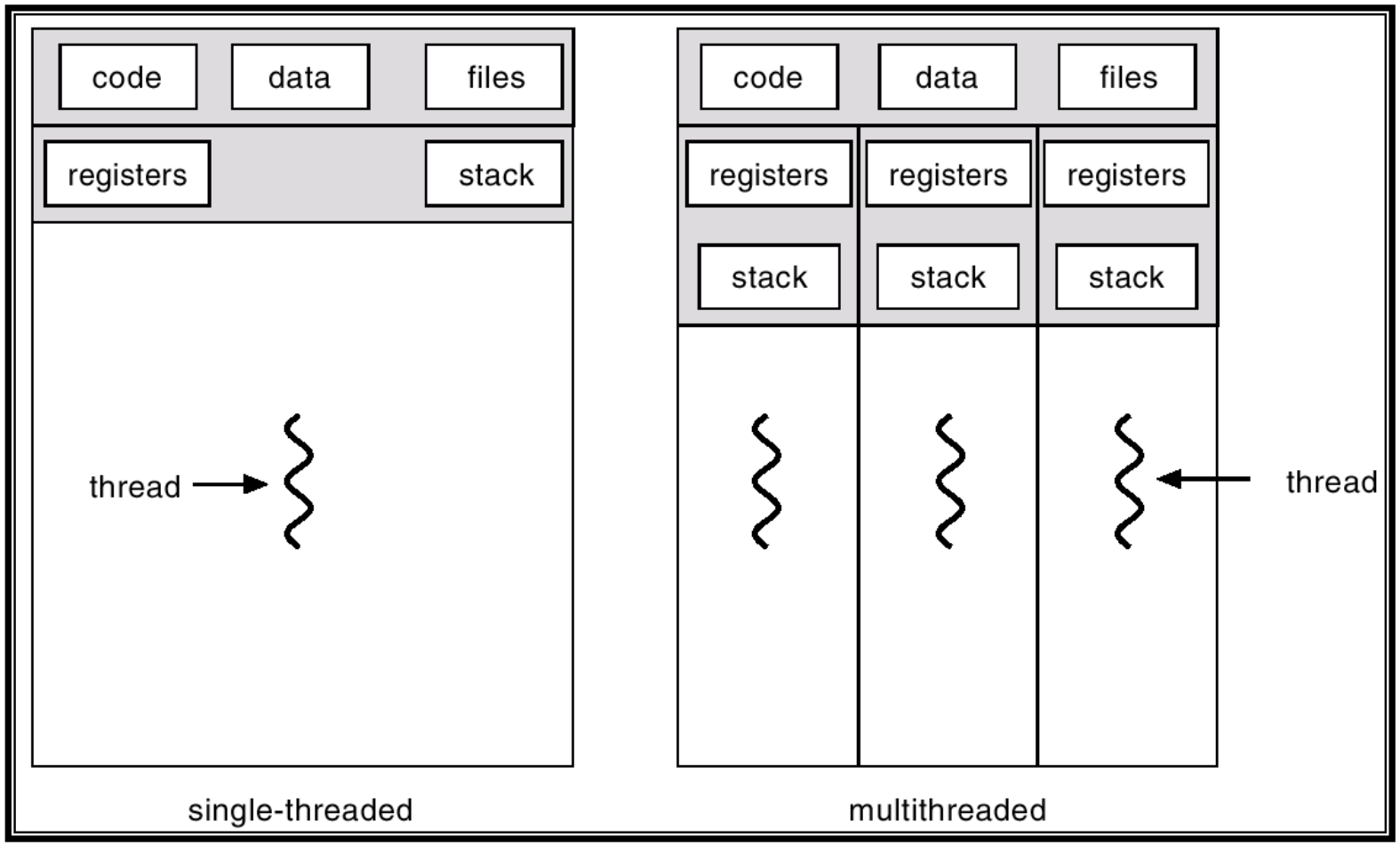
# Thread Advantage



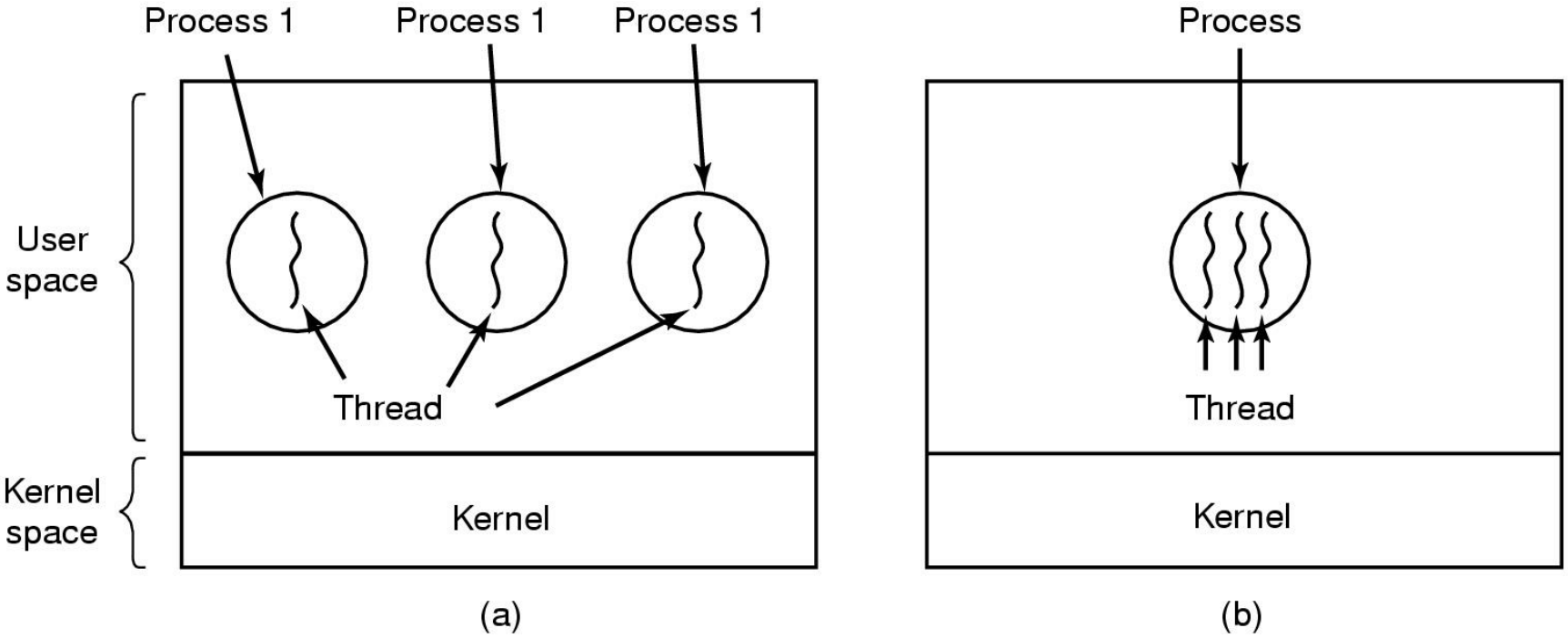
- Thread creation, short
- Thread exit, short
- Thread switch, short
- Communication of threads, simple

  (share the memory and file resource of process)

# Single and Multithreaded Processes



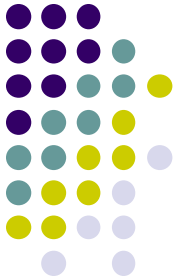
# The Thread Model (1)




(a) Three processes each with one thread

(b) One process with three threads

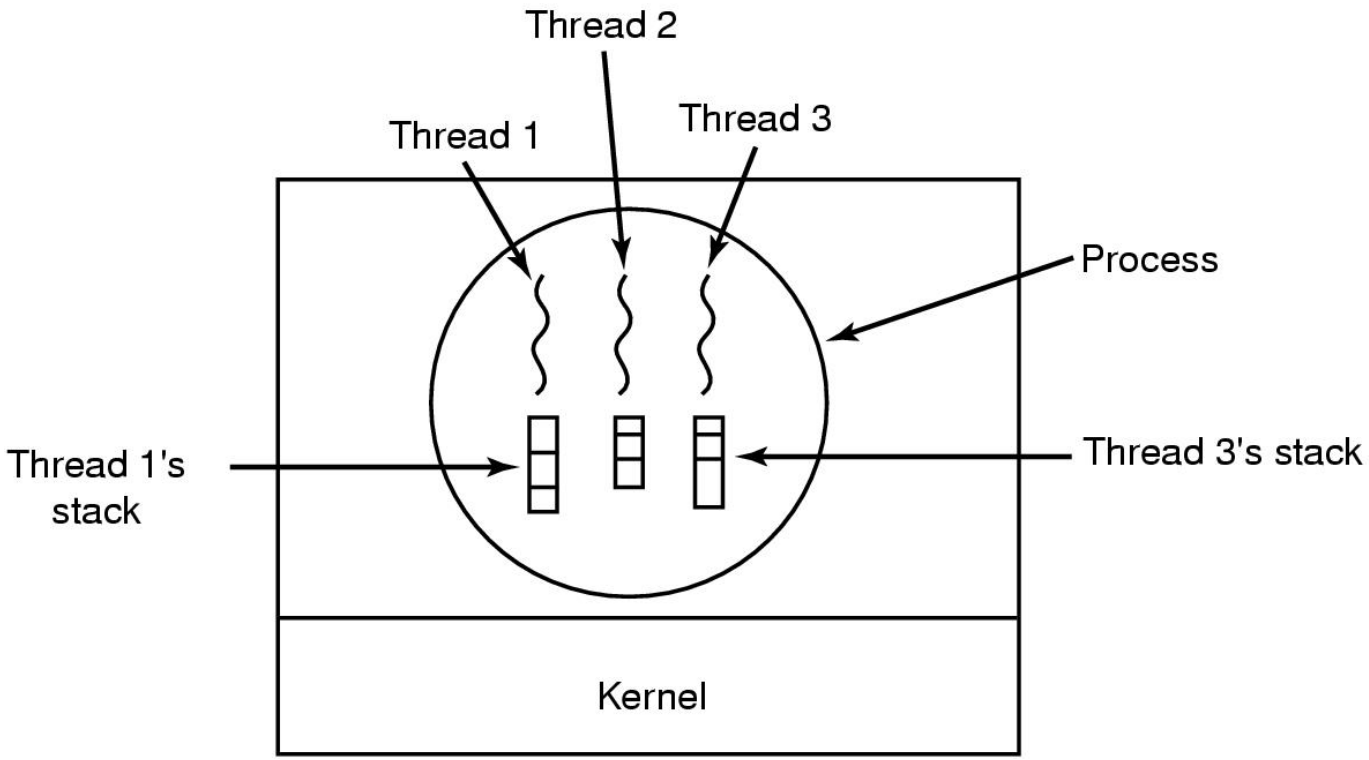
# The Thread Model (2)



| Per process items | Per thread items |
| --- | --- |
| Address space | Program counter |
| Global variables | Registers |
| Open files | Stack |
| Child processes | State |
| Pending alarms | |
| Signals and signal handlers | |
| Accounting information | |

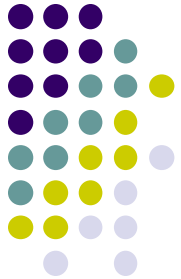
- Items shared by all threads in a process
- Items private to each thread

# The Thread Model (3)




Each thread has its own stack

# Thread Usage

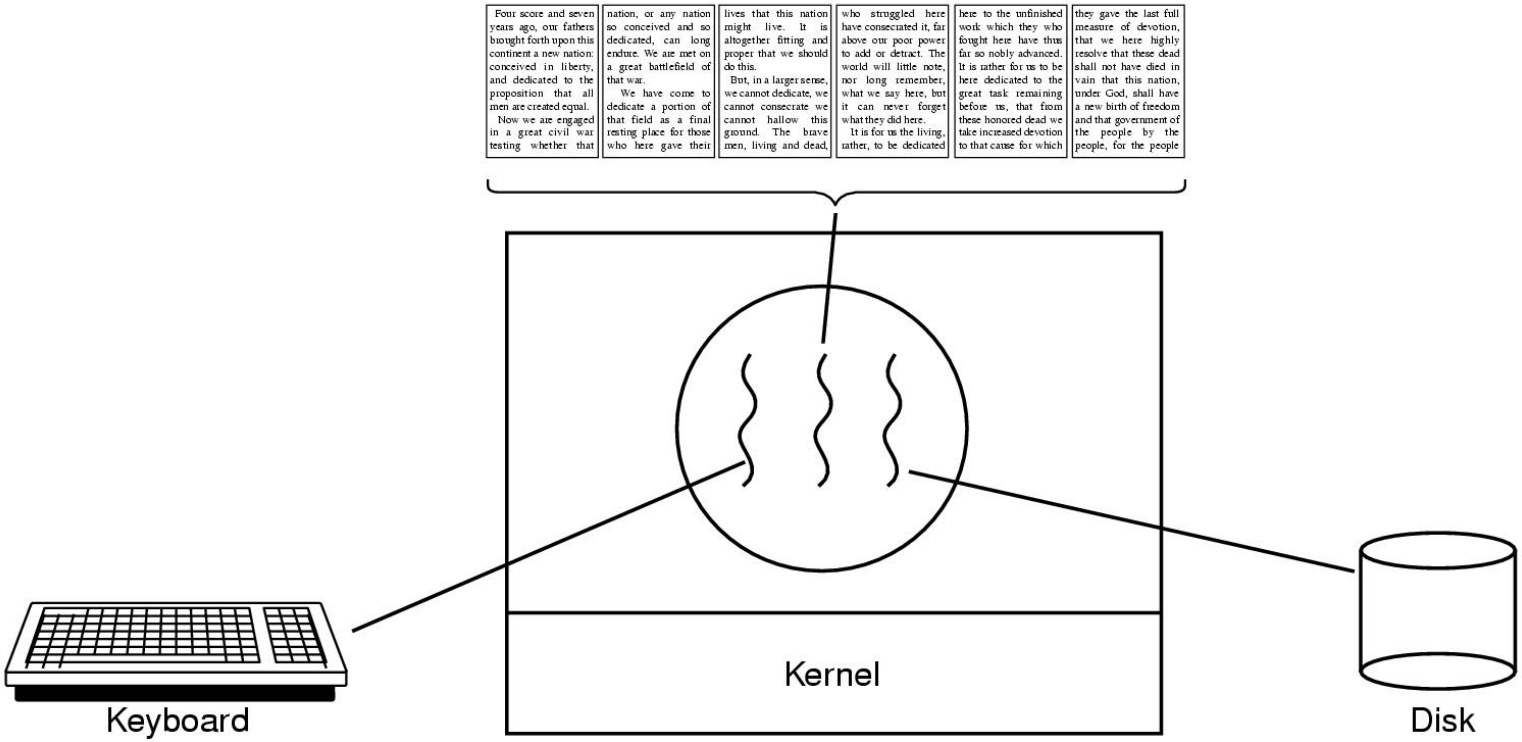


- Why do you use threads?
  - **Responsiveness**: Multiple activities can be done at same time. They can speed up the application.
  - **Resource Sharing**: Threads share the memory and the resources of the process to which they belong.
  - **Economy**: They are easy to create and destroy.
  - **Utilization of MP (multiprocessor) Architectures**: They are useful on multiple CPU systems.

- Example - Word Processor, Spreadsheet
  - One thread interacts with the user
  - One formats the document (spreadsheet)
  - One writes the file to disk periodically

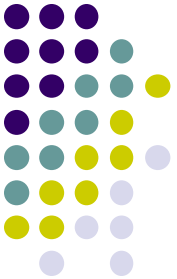
  Why not three processes?

# Thread Usage (1)



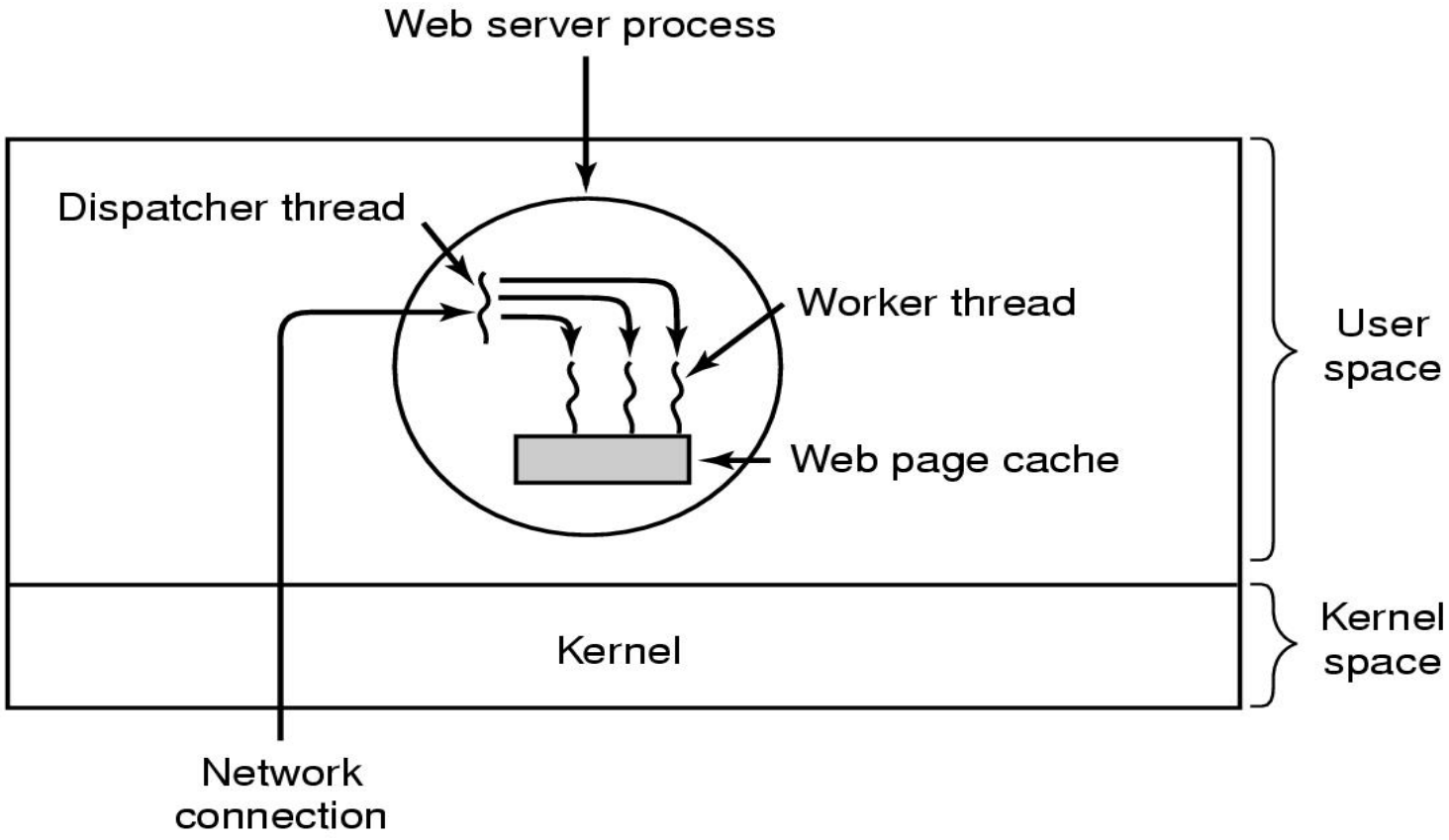

A word processor with three threads

# Thread Usage(2)



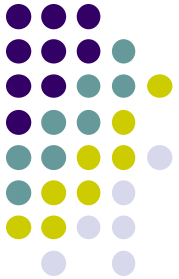
- Example – Web server
  - One thread, the dispatcher, distributes the requests to a worker thread
  - A worker thread handles the requests

# Thread Usage (2)




A multithreaded Web server

# Thread Usage (2)



```
while (TRUE) {
  get_next_request(&buf);
  handoff_work(&buf);
}
```

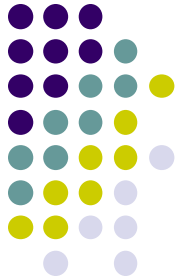
(a)

```
while (TRUE) {
  wait_for_work(&buf)
  look_for_page_in_cache(&buf, &page);
  if (page_not_in_cache(&page)
      read_page_from_disk(&buf, &page);
  return_page(&page);
}
```

(b)

- Rough outline of code for previous slide
  (a) Dispatcher thread
  (b) Worker thread

# POSIX Threads



| Thread call | Description |
|---|---|
| Pthread_create | Create a new thread |
| Pthread_exit | Terminate the calling thread |
| Pthread_join | Wait for a specific thread to exit |
| Pthread_yield | Release the CPU to let another thread run |
| Pthread_attr_init | Create and initialize a thread's attribute structure |
| Pthread_attr_destroy | Remove a thread's attribute structure |

Figure 2-14. Some of the Pthreads function calls

# POSIX Threads

```
#include <pthread.h>
#include <stdio.h>
#include <stdlib.h>

#define NUMBER_OF_THREADS     10

void *print_hello_world(void *tid)
{
        /* This function prints the thread's identifier and then exits. */
        printf("Hello World. Greetings from thread %d0, tid);
        pthread_exit(NULL);
}

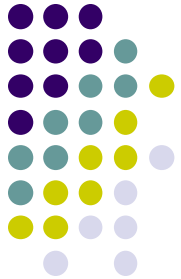
int main(int argc, char *argv[])
{
        /* The main program creates 10 threads and then exits. */
        pthread_t threads[NUMBER_OF_THREADS];
        int status, i;

        for(i=0; i < NUMBER_OF_THREADS; i++) {
                printf("Main here. Creating thread %d0, i);
                status = pthread_create(&threads[i], NULL, print_hello_world, (void *)i);

                if (status != 0) {
                        printf("Oops. pthread_create returned error code %d0, status);
                        exit(-1);
                }
        }
        exit(NULL);
}
```

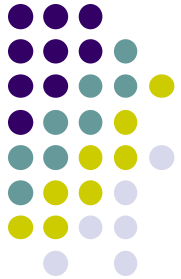
Figure 2-15. An example program using threads

# Implementation of Threads



- Three mechanisms
  - Implementing threads in user space (ULT)
  - Implementing threads in kernel space (KLT)
  - Hybrid Implementations

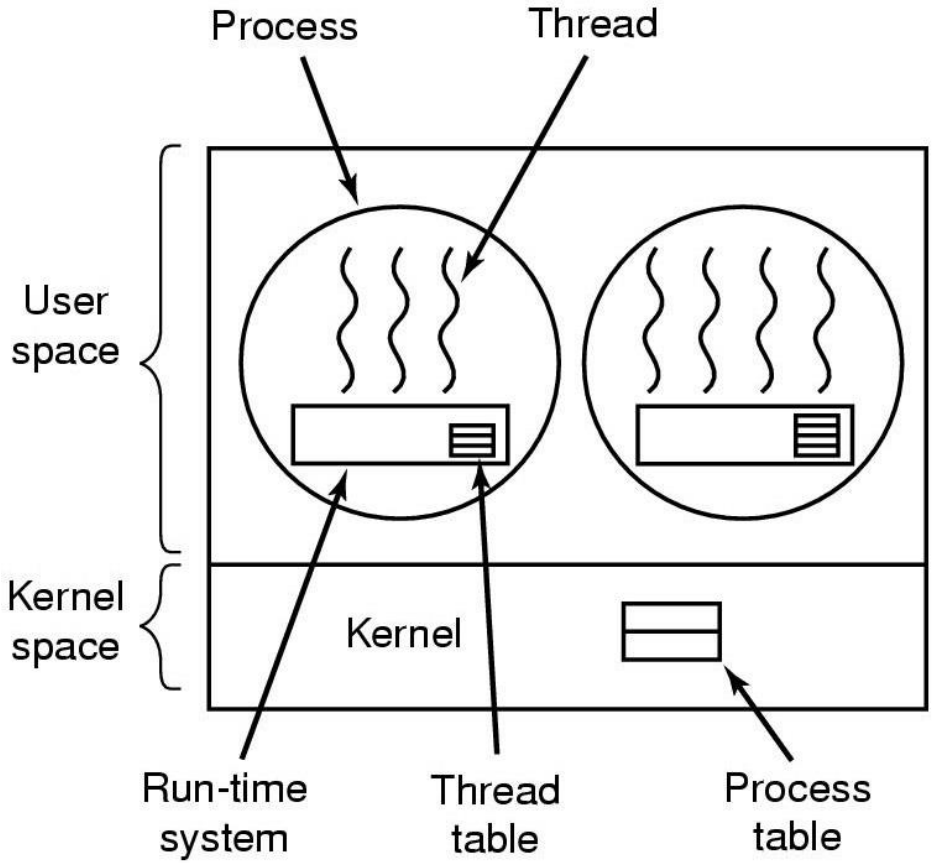
# User Threads(ULT)



- The threads package entirely in user space, kernel doesn’t know thread
- Threads management done by user-level threads library
- Thread switch don’t need kernel privilege, user-level threads are fast to create and manage
- Problem: If the kernel is single-threaded, then any user-level thread performing a blocking system call will cause the entire process to block
- Examples of threads library

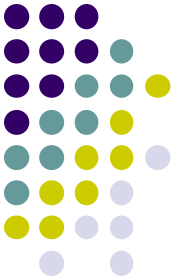
  - POSIX *Pthreads*
  - Mach *C-threads*
  - Solaris UI-*threads*

# Implementing Threads in User Space




A user-level threads package

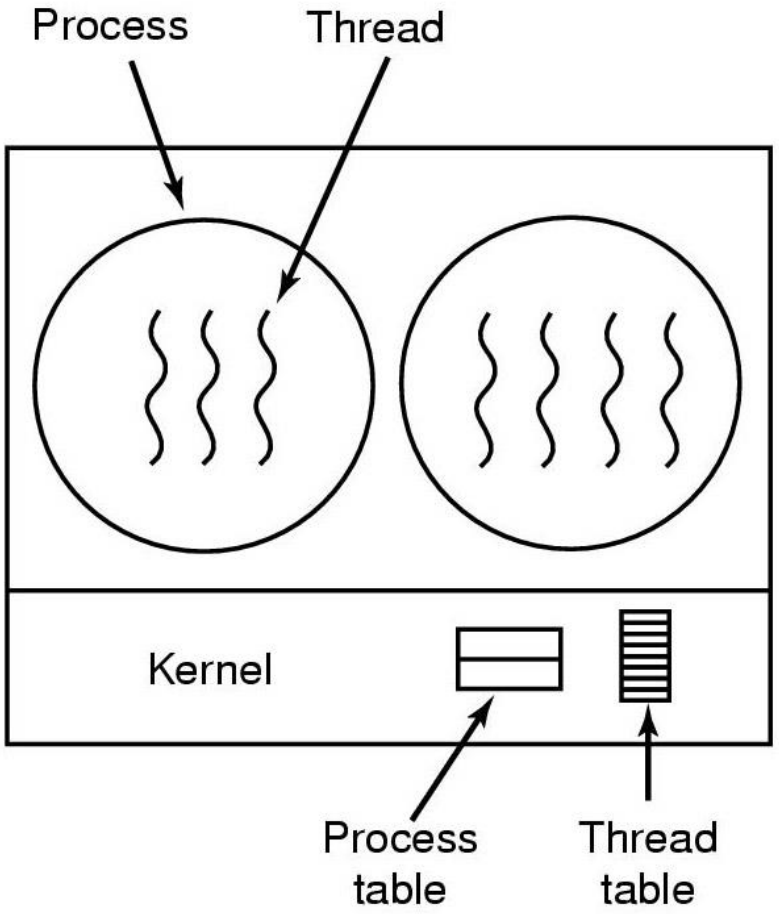
# Kernel Threads(KLT)



- Supported by the Kernel:
  The kernel performs thread creation, scheduling, and management in kernel space
- No thread library, kernel provide API
- Kernel maintain context of process and thread
- Thread switch need kernel
- Thread is the basic unit of Scheduler

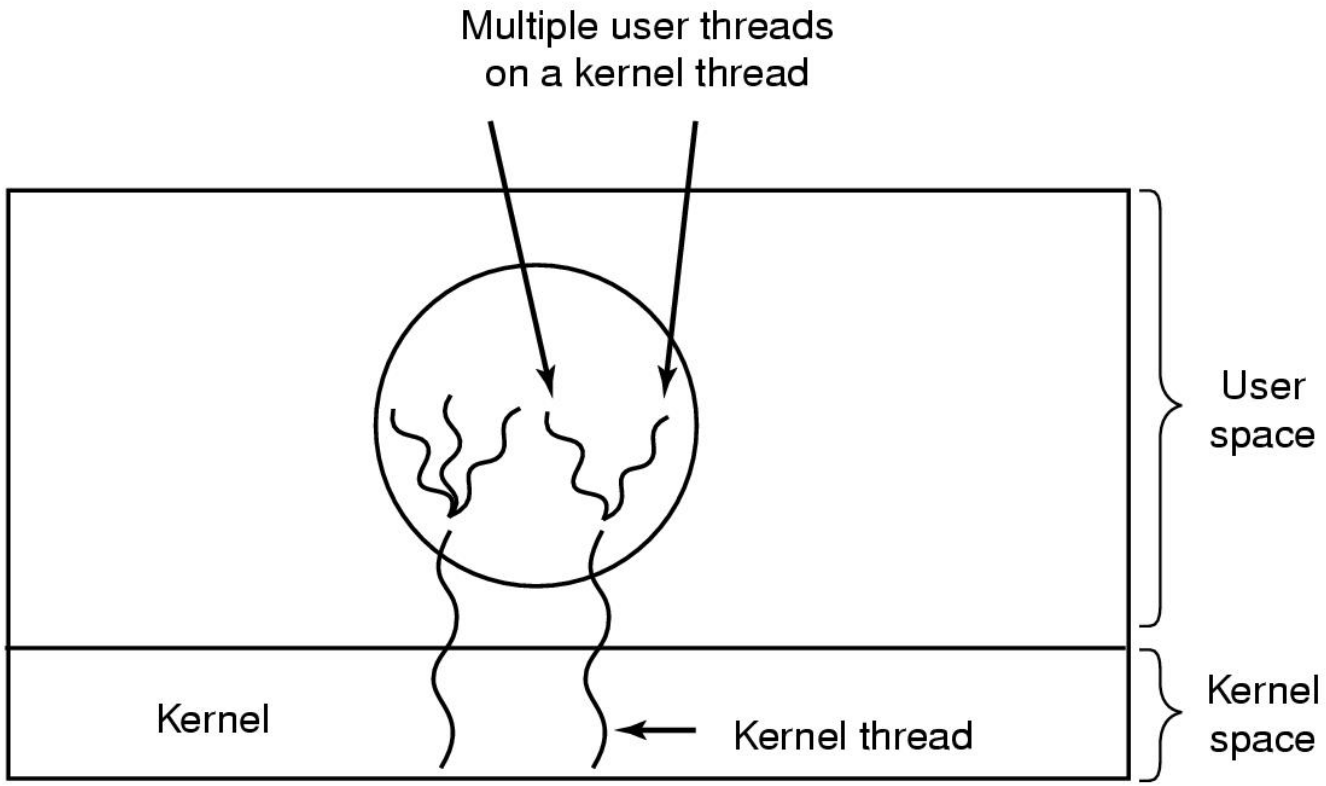
- Disadvantage: high cost
- Examples
  - *Windows 95/98/NT/2000/XP*

# Implementing Threads in the Kernel



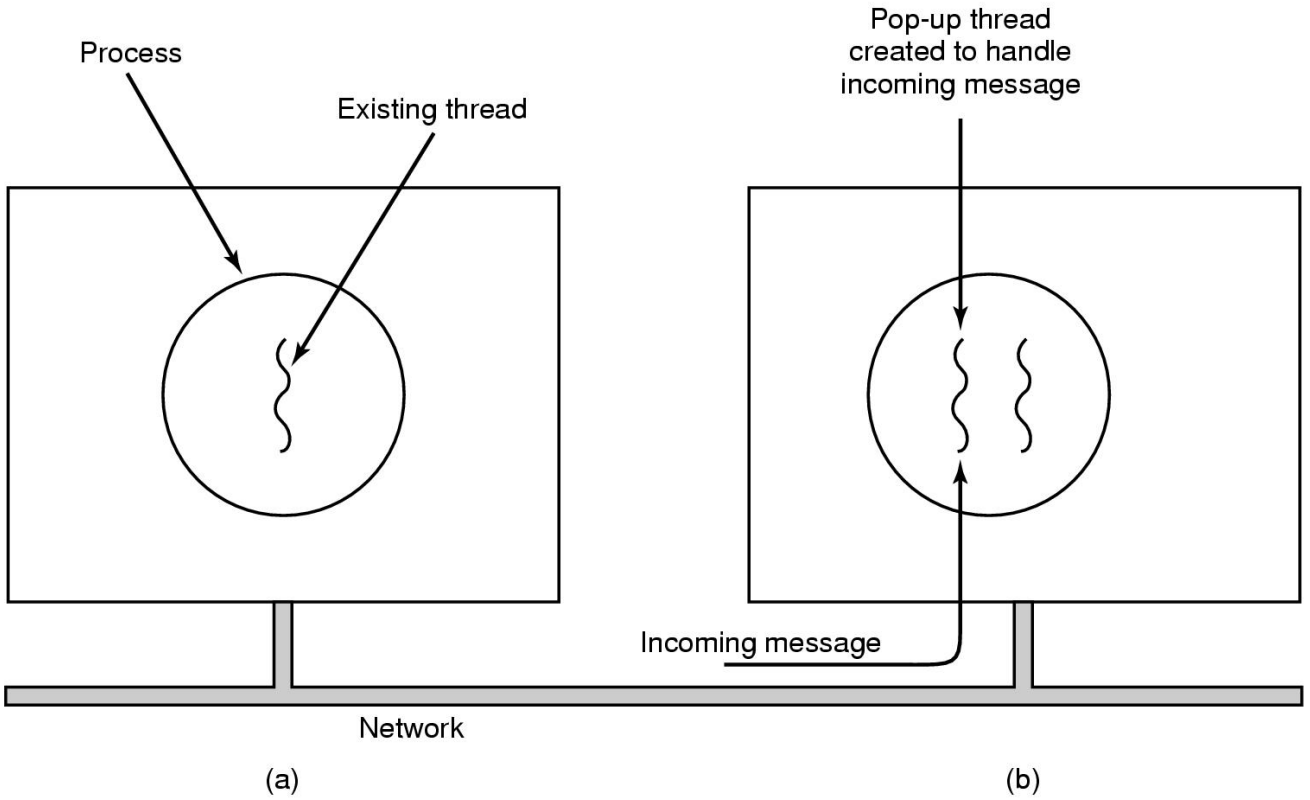

A threads package managed by the kernel

# Hybrid Implementations



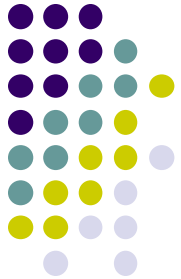

Multiplexing user-level threads onto kernel-level threads (SUN solaris)

# Pop-Up Threads




- Creation of a new thread when message arrives
  (a) before message arrives
  (b) after message arrives

# Summary(2)



- What is thread? Why need threads?
- Difference between process and thread
- Thread Usage
- Thread Implementations and their tradeoffs
  - User-level
  - Kernel-level
  - Hybrid
  - Pop-up threads

# Thank you!

